Deployment and Operations for Software Engineers

Len Bass and John Klein

Preface

Three recent trends are dramatically extending the responsibilities of software engineers and, hence, the requisite skill set. The widespread adoption of the cloud is one of these trends Software engineers need to be familiar with virtualization both of hardware and operating systems, with how networks work, and with managing failure in the cloud, scaling, and load balancing.

Driven by the adoption of the cloud, release cycles have shortened dramatically. It is no longer sufficient to release once a quarter. Monthly, weekly, daily, and hourly releases are common with the release time dependent more on business considerations than on technical ones. Short release cycles are supported by the migration to microservices, the proliferation of tools for a deployment pipeline, and by the growth of container orchestration tools. Yet rapid release cycles also depend on architectural decisions and disciplined use of the available tools.

Today, a software engineer is often expected to be responsible for their portion of the system during creation, deployment, and execution. This, together with the architecture of the cloud, requires an engineer to understand operations concepts such as business continuity and incident handling. They also must understand logging, monitoring, and alerts.

A software engineer, typically, learns this material on the job, requiring much work going through blogs, tutorials, and system documentation. This is a difficult and time-consuming process. Some organizations have training courses for new hires where an organization's specific processes and tools are covered. Usually, however, these courses do not provide a broad context for the processes and tools.

This book is aimed at software engineers (and prospective software engineers) who are juniors/seniors or first-year graduate students in computer science or software engineering. It provides the context within which modern engineers work and the skills necessary to operate within that context.

This book also supports the teaching of this material by an instructor. Each chapter discusses the theory associated with its topics and then has hands-on exercises to apply the theory. There are also discussion questions that can be answered in a classroom setting or in discussion groups to enable the students to gain a better understanding of the theory and its implications. The modular

nature of the presentation of the material makes it easy to skip sections that cover familiar material.

Our goal in this book, admittedly ambitious, is to distill the knowledge required of modern software engineers beyond programming languages and a three-tiered architecture. We cover six major topics: Virtualization and the cloud, networks and distribution, microservice architecture, deployment, security, and operations including business continuity and monitoring. We make no claim that the reader of this book will emerge as an expert in any of these areas, but they will have a basic understanding of the topics and some hands-on exercises to cement this understanding. This provides the basis for an engineer to become productive in an organizational setting more quickly.

Since the beginning of computer science education, there has been a debate about the extent to which a student should understand what is below a given level of abstraction. Higher level languages mask machine language–how much does the student need to know about machine language? Optimizing compilers mask certain reorganizations of computation–how much does the student need to know about these reorganizations? Java masks memory allocation–how much does the student need to know about memory allocation? And so on. In almost every case, the answer has been some, but not too much.

This is the view we are taking in this book. The software engineer needs to know something about virtualization, networking, the cloud, and the other topics we cover, but they do not need to know the gritty details. Computer science and software engineering curricula have courses in distributed networks (the cloud), security, and software architecture. These courses go into more detail than the typical software engineer needs. This leaves a gap that we are attempting to fill with this book. Enough knowledge of these topics for the software engineer but not so much knowledge that an engineer is overwhelmed.

Introduction to Part 1: The Platform Perspective

You have just graduated and started a new job and been given your first assignment. Your company is running a promotion and you will write a service that calculates the discount based on a set of rules involving other customer purchases and the organization's loyalty program. You breathe a sigh of relief; it doesn't look too hard, maybe a little complicated to understand the discount rules, but certainly manageable. Then your manager says: "You should probably set the scaling rules at 5 seconds, allocate a new VM at 80% utilization, and keep your service stateless. Also, when you containerize your service, use our private Docker repository and not the public Docker Hub. After you generate SSH keys, send your public key to Mary and she'll put it on our DMZ servers. I've got to run–over the weekend, we had a spike in IDS alarms that I need to review."

Huh? If these instructions are bewildering, then Part 1 is for you. Part 1 describes the cloud as a platform for development and focuses on those things that are out of your control as a developer.

Today's developers need to understand much more than programming languages and business rules. The widespread adoption of cloud computing means that your organization is probably utilizing a large distributed network for its basic computing platform. This platform can be public or private, but in either case a developer needs to understand enough about virtualization and distributed computing to operate effectively in this environment.

A cloud data center has tens of thousands of computers communicating over a network and is accessible externally via the internet. An application running in the cloud ideally acquires the resources that it needs (CPU, network, disk) for only as long as they are needed. These resources–virtual machines or containers–run on physical machines. They are either allocated automatically and dynamically by the application or reserved, in advance, by the application developer. Thus, in Chapter 1 Virtualization we introduce virtualization. Hypervisors allow virtual machines to share physical hardware. More recently, containers that virtualize the operating system have been in widespread use.

We introduce containers in Chapter 1 Virtualization and discuss them in more detail in Chapter 4 Container Management.

The physical and virtual machines in the cloud communicate over networks–both internal to the cloud and external using the internet. Communication over networks transmits data using messages. Two concerns immediately arise when using messages as the communication mechanism–how the messages are delivered to the correct recipient and what protocol is used to deliver them. By "correct recipient," we mean not only the virtual machine intended as the target but also the service within that machine. Furthermore, networks have structure and sub-structure. Chapter 2 is a discussion of networks.

With virtualization and networks as building blocks, Chapter 3 discusses the cloud. We will see how the cloud allocates resources. When tens of thousands of computers are involved, it is inevitable that some will fail, and handling failures permeates the design of services that run on the web. In Chapter 3, we introduce the possibility of failure, which will be elaborated in future chapters. As workloads change, virtual machines become overloaded; however, the cloud provides technology to handle overload by distributing the load over multiple instances of a virtual machine. These instances can be allocated dynamically: load balancers and autoscaling manage the creation and destruction of instances. Multiple service instances and multiple clients introduce the problems of state management and distributed coordination. We discuss the motivation for distributed systems and explain how the complications of distributed coordination systems are hidden behind understandable interfaces.

In chapter 4 Container Management, we return to containers. Containers are managed by different mechanisms than virtual machines, although there is a relation. Container images can be stored in a repository, container instances can be scaled similarly to virtual machines, and containers are used in *serverless architecture*. All these topics are discussed in this chapter.

Finally, in Part 1 we discuss the basic concepts of security. In today's environment, security is a concern of every software engineer. Some elements of securing systems are provided by the platform and the history of the internet. Chapter 5 discusses these elements, starting with a framework to separate security concerns. We provide a brief overview of cryptography, and present several protocols and systems that use elements of cryptography to create robust, secure solutions. We finish the chapter with a discussion of

technology used to detect cyberattacks against your virtual computers and your networks.

At this point, if you are asking yourself "what about DevOps", we will do a thorough job of covering the tool aspects of DevOps, with some here in Part 1 but most in Part 2. The cultural and organizational aspects of DevOps are outside the scope of this book.

Chapter 1 Virtualization

Much computing these days is done on virtual machines. Even systems such as automobiles rely heavily on virtualization. When you finish this chapter and do the exercises, you will understand

- The goals of virtualization
- How virtualization works
- The distinction between a virtual machine instance and a virtual machine image
- The distinction between virtualizing hardware and virtualizing operating systems.

Sharing and Isolation

In the 1960s, the computing community was frustrated by the problem of sharing resources such as memory, disk I/O channels, and user input devices on one physical machine among several independent applications. The inability to share resources meant that only one application could be run at a time. Computers at that time cost millions of dollars and most applications used only a fraction of the available resources, so this constraint had a significant effect on computing costs.

Several mechanisms emerged to deal with sharing. The goal of these mechanisms was to isolate one application from another, while still sharing resources. Isolation allows developers to write applications as if they are the only ones using the computer, while sharing resources allows multiple applications to run on the computer at the same time. Since the applications are sharing one physical computer with a fixed set of resources, there are limits to the illusion that isolation creates. If, for example, one application consumes all the CPU resources, then the other applications cannot execute. For most purposes, however, these mechanisms, with some modification, have been sufficient.

Sidebar: Terminology

The terminology we will be using throughout this book differentiates the terms application, client, and service.

A service is a coherent collection of functionality. Some services are exposed to end users while others are not. We discuss a particular type of restriction to services, called microservices, in depth in Chapter 6.

A client of a service is another service that uses the original service. An application presents a collection of services to an end user. The services that are visible to end users are clients for other services which may be, in turn, clients for still more services.

We will refer to the software that you develop as a service. These services and the applications that they support are under your control, at least within the constraints of the organization that you are working within.

Finally, there is software that is not under your control. These are also services (each provides some collection of functionality), and we will label each of these with an adjective that describes its function, such as "operating system service," "storage service," or more generally, "infrastructure service."

Implicit in these definitions is a distinction between an end user and a client. An end user is a human, possibly an administrator, and a client is a piece of software

We now describe the resources that are shared. Next, we will describe the isolation and sharing mechanisms. There are four such resources:

1. Central processing. Modern computers have multiple central processing units, or CPUs (and each CPU can have multiple processing cores), and may have one or more graphics processing units (GPUs) or other special-purpose processors, such as a tensor processing unit (TPU).
2. Memory. The physical computer has a fixed amount of physical memory.
3. Disk. Disks provide persistent storage for instructions and data, across reboots and shutdowns of the computer. A physical computer has one or more attached disks, each with a fixed amount of storage capacity.
4. Network connection. Networks were a later addition to the set of resources to be shared among applications, emerging in the 1970s.

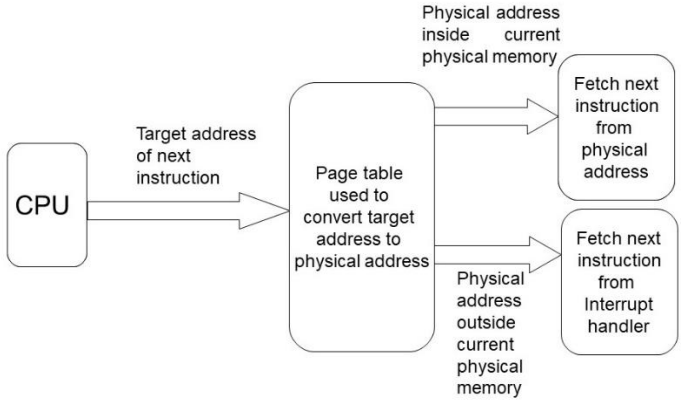
Today, every non-trivial physical computer has one or more network connections through which all messages pass. These include both outbound messages and inbound messages.

Now we turn to the isolation and sharing mechanisms.

- Processor sharing is achieved through the thread-scheduling mechanism. The scheduler selects and assigns a thread to an available processor, and that thread maintains control until the processor is rescheduled. No other thread can gain control of this processor without going through the scheduler. Rescheduling occurs when the thread yields control of the processor, when a fixed time interval expires, or when an interrupt occurs.
- As applications grew, all the code and data would not fit into physical memory, and virtual memory technology emerged. This memory management hardware partitions a process's address space into *pages*, and swaps pages between physical memory and disk storage as needed. The pages that are in physical memory can be accessed immediately, and other pages are stored on the disk until they are needed. A simplified view of virtual memory is that every memory reference, either a fetch or a store, is checked to see whether the referenced memory location is currently in physical memory. If it is, then the reference proceeds. If it is not, a page-fault interrupt is generated and the relevant page is fetched from disk and placed into memory. Then the memory reference is repeated. This technology evolved to tag pages with the process ID, providing isolation between processes in a physical machine. Virtualization extends this tagging to isolate memory pages between VMs in a physical machine, and between processes within each VM.

  Figure 1.1 presents a logical view of this process. Modern hardware provides a number of optimizations that we will not go into.

Figure 1.1: Logical View of How Virtual Memory Is Managed on a Physical Machine



- Disk sharing and isolation are achieved using several mechanisms. First, the physical disks can be accessed only through a disk controller that ensures that the data streams to and from each thread are delivered in sequence. Also, the operating system may tag executing threads and disk content such as files and directories with information such as a userid and group, and restrict visibility or access by comparing the tags of the thread requesting access and the disk content.
- Network isolation is achieved through the identification of messages. Every virtual machine (VM) has an address used to identify messages to or from that VM. The hypervisor implements network infrastructure within the physical machine that allows VMs to share and isolate use of the physical network interface using the same approaches and protocols used for networking between physical machines. In the next chapter, we discuss IP addresses and the role of the hypervisor.

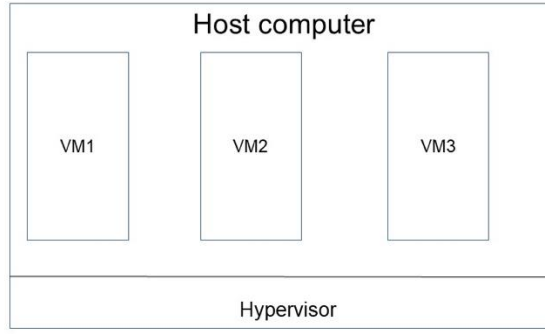
Now that we have seen how the resource usage of one application can be isolated from the resource usage of another application, we turn to a discussion of virtual machines (VMs), which implement and combine these mechanisms to allow the execution of multiple simulated, or virtual, computers in a single physical computer. You may encounter another common usage of the term “virtual machine”–the Java virtual machine (JVM)–which is not

included in this book. Using our terminology, we would say that a JVM is a specialized service that executes Java bytecode.

1.2 Virtual Machines

Figure 1.2 shows several VMs residing in a physical computer. The physical computer is called the "host machine" and the VMs are called "guest machines." Figure 1.2 also shows a hypervisor which is an operating system for the virtual machines. This hypervisor runs directly on the physical computer hardware and is often called a *bare-metal* or *Type 1* hypervisor. The VMs that it hosts implement applications and services. Bare-metal hypervisors typically run on a data center or cloud.

Figure 1.2: Bare-Metal Hypervisor and VMs



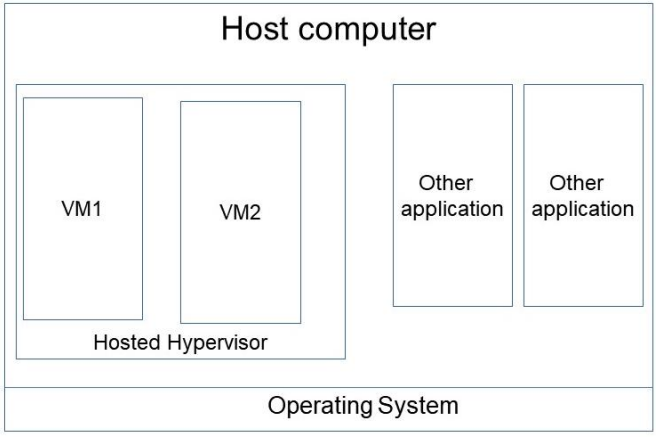
Look again at Figure 1.1 in the light of multiple VMs on a host computer. Since the operating system in each VM manages the page table for the software executing on the VM, each VM has its own page table. The hardware must translate a target address first to the correct VM and then to the page table within that VM. Modern hardware is designed with knowledge of virtualization and its mechanisms. The hardware uses multiple techniques and optimizations to enable the CPU to access the correct memory location. We will not go into detail about these techniques.

There is another type of hypervisor, called a *hosted* or *Type 2* hypervisor, which is shown in Figure 1.3. In this case, the hypervisor runs as a service on top of a host operating system, and the hypervisor in turn hosts one or more VMs. Hosted hypervisors are typically used on desktop or laptop computers. They allow users to run and test applications that are not compatible with the

computer's host operating system (e.g., run Linux applications on a Windows computer). They are also used to replicate a production environment on a development computer even if the operating system is the same on both the development computer and the production environment.

A hypervisor requires that its guest VMs use the same instruction set as the underlying physical CPU–the hypervisor does not translate or simulate instruction execution. For example, if you have a VM for a mobile or embedded device that uses an ARM processor, you cannot run that virtual machine on a hypervisor that uses an x86 processor. There is another technology, related to hypervisors, for cross-processor execution, called an *emulator*. An emulator reads the binary code for the target or guest processor and simulates execution of guest instructions on the host processor. The emulator often also simulates guest I/O hardware devices. For example, the open source QEMU emulator[1] can emulate a full PC system, including BIOS, x86 processor and memory, floppy disk drive, sound card, and graphics card.

Figure 1.3: Hosted Hypervisor



Both hosted/Type 2 hypervisors and emulators allow a user to interact with the guest application through an on-screen display, keyboard, and mouse/touchpad. Developers working on desktop applications or working on

[1] https://www.qemu.org

specialized devices, such as mobile platforms or devices for the Internet of Things (IoT), may use a hosted/Type 2 hypervisor and/or an emulator as part of their build/test/integrate toolchain. However, we will not discuss emulators in this book and will be focused on bare-metal, or Type 1, hypervisors and the VMs that they host.

A hypervisor performs two main functions: (1) managing the code running in each VM, and (2) managing the VMs themselves. To elaborate,

1. Code that communicates outside the VM by accessing a virtualized disk or network interface is intercepted by the hypervisor and executed by the hypervisor on behalf of the VM. This allows the hypervisor to tag these external requests so that the response to these requests can be routed to the correct VM.

   The response to an external request to an I/O device or the network is an asynchronous interrupt. This interrupt is initially handled by the hypervisor. Since multiple VMs are operating on a single physical host machine and each VM may have I/O requests outstanding, the hypervisor must have a method for forwarding the interrupt to the correct VM. This is the purpose of the tagging mentioned above.

   One additional type of instruction that is neither a normal instruction nor an external request is a *system call*. In this case, code operating in user mode inside the VM asks its internal operating system for a service and this, frequently, involves changing the mode of the code being executed in the VM to kernel mode. Depending on the hardware architecture, a system call may generate an interrupt. In this case the interrupt is initially fielded by the hypervisor and forwarded to the appropriate VM, just as in the case of an asynchronous interrupt from an I/O device or the network.

2. VMs must be managed. For example, they must be created and destroyed, among other things. Managing VMs is a function of the hypervisor. The hypervisor does not decide on its own to create or destroy a VM. It acts on instructions from a user or, more frequently, from a cloud superstructure. We will explore how this works in Chapter 3 The Cloud.The process of creating a VM involves loading a VM image, and we discuss this in the next section.

> In addition to creating and destroying VMs, the hypervisor monitors them. Health checks and resource usage are part of the monitoring. The hypervisor is also in the defensive security perimeter of the VMs with respect to attacks.
>
> Finally, the hypervisor is responsible for ensuring that a VM does not exceed its resource utilization limits. Each VM has limits on CPU utilization, memory, and disk and network I/O bandwidth. Before starting a VM, the hypervisor first ensures that there are sufficient physical resources available to satisfy that VM's needs, and then the hypervisor enforces those limits while the VM is running.

A VM is booted just as a *bare metal* physical machine is booted. When the machine begins executing, it automatically reads a special program called the *boot loader* from a disk drive,[2] either a disk drive internal to the computer or a disk drive connected through a network. The boot loader reads the operating system code from disk into memory, and then transfers execution to the operating system. In the case of a physical computer, the connection to the disk drive is made during the power up process. In the case of the VM, the connection to the disk drive is established by the hypervisor when it starts the VM. The next section about VM images discusses this in more detail.

From the perspective of the operating system and service software inside a VM, it appears that the software is executing inside of a bare physical machine. The VM provides a CPU, memory, I/O devices, and a network connection. In **Error! Reference source not found.**, we will see that each VM also has its own IP address. In fact, it is quite difficult for an operating system to detect whether it is running in a VM or physical machine–for example, it can look for evidence in log files, but the detection approaches are ad hoc and specific to each hypervisor.

All these concerns mean that the hypervisor is a complicated piece of software. One concern with virtual machines is the overhead introduced by the sharing and isolation needed for virtualization. That is, how much slower does a service run on a virtual machine, compared to running directly in a bare physical machine? This turns out to be a complicated question since it depends on the

[2] We use the term "disk drive" to refer to either a rotating hard disk drive (HDD) device or a solid-state disk drive (SSD) device, recognizing that SSDs have neither disks nor any moving parts to drive.

characteristics of the service and on the virtualization technology used. For example, services that perform more disk and network I/O incur more overhead than services that do not share these host resources. Virtualization technology is improving all the time, but overheads of around 10% have been reported by Microsoft on their Hyper-V hypervisor[3].

1.3 VM Images

Just as a physical computer without software merely consumes power and generates heat, a virtual machine without software is not very useful. A physical computer loads software by reading directly from a built-in disk drive, an external disk drive, or a storage access network (SAN).[4] The same is true for a VM, except that instead of direct hardware or network access it accesses its potential sources of software through the hypervisor.

The first thing to realize is that the software to be loaded into the machine (physical or virtual) exists as a collection of bits on some media. A sequence of bits can be interpreted as data, an instruction, or an address. The boot process treats the bits first as a data stream and then as a set of instructions, possibly with embedded data. Booting the machine copies the boot bits into memory and executes the bits as instructions. A simplified view of the boot process is

1. Connect to a disk drive.
2. Read bits from a pre-determined location (e.g., the *master boot record*), and put them memory.
3. Transfer execution to the location where the boot record was loaded and execute those bits as the instructions of the *boot loader* program. This program reads more bits into memory.
4. Execute those newly loaded bits as instructions of the operating system.

For physical machines, step 1 is performed by hardware when the machine is reset or power is turned on, and step 2 is performed by the BIOS firmware. For a VM, steps 1 and 2 are performed by the hypervisor. Although some hypervisors require special boot loader programs, Steps 3 and 4 are essentially the same for both physical and virtual machines.

[3] https://docs.microsoft.com/en-us/biztalk/technical-guides/system-resource-costs-on-hyper-v
[4] https://en.wikipedia.org/wiki/Storage_area_network

We call the contents of the disk drive that we boot from an *image.* This image contains the bits that represent the instructions and data that make up the software that we will run (i.e., operating system and services). The bits are organized into files and directories according to the file system used by your operating system. The image also contains the boot load program, stored in its pre-determined location.

There are several ways to create a new image. One common approach is to find a machine that is already running the software we want and make a snapshot copy of the bits in that machine's memory.

Another approach is to start from an existing image and add additional software. There are repositories of machine images (usually containing open-source software) providing a variety of minimal images with just OS kernels, other images that include complete applications, and everything in between. These provide efficient starting points and support quickly trying out a new package or program. However, there are some issues when pulling down and running an image that you (or your organization) did not create: there are relatively minor issues, such as inability to control the versions of the OS and software, and there are more significant issues related to security. The image may have software that contains vulnerabilities or that is not configured securely, or worse, the image may include malware. In Chapter 11 Secure Development, we discuss security issues related to the download of software from the web.

A third approach, creating an image from scratch, is a bit more complicated. First consider creating an image on a physical machine–you may have done this yourself if you have a new laptop computer. You start by obtaining *install media* for your chosen operating system. This is a bootable CD, DVD, or USB memory stick that contains a program to create the image you want. You boot your new machine from the install media, and it formats the machine's disk drive, copies the operating system onto the drive, and adds the boot loader in the pre-determined location. You remove the install media device, restart your machine, and it boots from your newly created image. You can then add whatever services you want to the image, and maybe make a snapshot copy to use on other machines.

You can follow a similar approach to create an image for a virtual machine. Here, start by using a Type 2 or hosted hypervisor, so that you can interactively

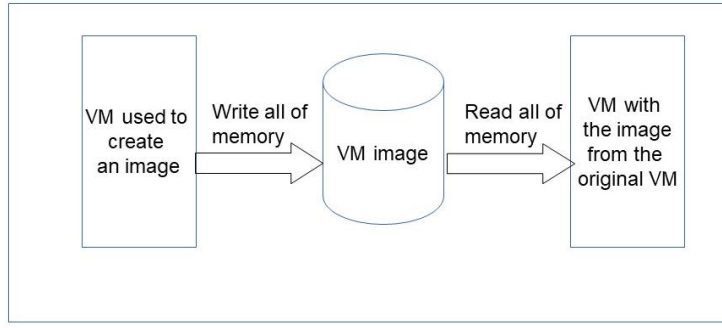
control the hypervisor and have a GUI for the VM. You command the hypervisor to start a new VM with an empty virtual disk drive and boot the VM from your install media. The installer program copies the operating system files to the VM's virtual disk drive. You may need to have the hypervisor add its customized boot loader to your virtual disk (as mentioned earlier, some hypervisors require a special boot loader). You can then make a snapshot copy of the bits on your virtual disk, and use that image to boot other VMs using Type 1 or Type 2 hypervisors.

This manual approach to creating a VM image is easy to understand but requires a person to attend to each of the tasks. As we will see in Chapter 11 Secure Development, you need to update your images frequently as patches become available for your operating system and other software. To operate efficiently, you need to automate the creation of new images. We will discuss this in Chapter 7 Managing System Configurations.

You might have noticed that we did not mention installing services on the newly created image. While you could easily install services when creating an image, this would lead to a unique image for every version of every service. Aside from the storage cost, this proliferation of images becomes difficult to keep track of and manage. It is customary to create images that contain only the operating system and other essential programs, and then add services to these images after the VM is booted, in a process called *configuration*.

We summarize the process of creating and loading a virtual machine image in Figure 1.4, which shows a disk file being generated from one VM and subsequently loaded by the hypervisor into another VM.

Figure 1.4: The Process of Creating, Saving, and Loading an Image



1.4 Containers

VMs solve the problem of sharing resources and maintaining isolation that was posed in the 1960s. However, VM images can be large, and transferring VM images around the network is time consuming. Suppose you have an 8 GB(yte) VM image. You wish to move this from one location on the network to another. In theory, on a 1 Gb(it) per second network, this will take 64 seconds. However, in practice a 1 Gbps network operates at around 35% efficiency. Thus, transferring an 8 GB VM image will take around 3 minutes. There are approaches that can reduce this transfer time, but the result is still on the order of minutes. After the image is transferred, the VM must boot the operating system and start your services, which takes still more time.

Containers are a mechanism to maintain the advantages of virtualization while reducing image transfer time and startup time.

Re-examining Figure 1.2, we see that a VM executes on virtualized hardware under the control of the hypervisor. In Figure 1.5, we see several containers operating under the control of a container engine which, in turn, is running on top of a fixed operating system. By analogy with VMs, containers run on a virtualized operating system. Just as all VMs on a physical host share the same underlying physical hardware, all containers share the same operating system kernel (and through the operating system, they share the same underlying physical hardware). The operating system can be loaded either onto a bare-metal physical machine or a virtual machine.

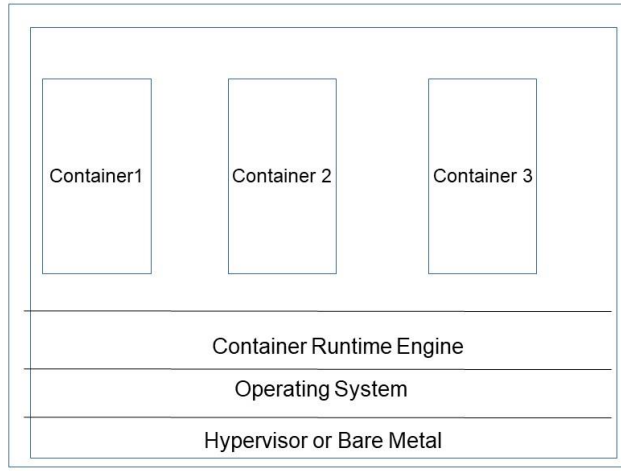
Container engines use several Linux features to provide isolation while sharing the operating system. Linux control groups set resource utilization limits, and Linux namespaces prevent a container from seeing other containers.

This sharing of the operating system gives us one source of performance improvement when transferring images. As long as the target machine has a container engine running on it, there is no need to transfer the operating system as part of the container image. Since modern operating systems are on the order of 1 GB(yte) in size, this saves a substantial amount of time.

The second source of performance improvement is the use of layers in container images. To understand this, we describe how a container image is constructed. We illustrate the construction of a container to run the *LAMP*

stack, and we will build the image in layers. LAMP is Linux, Apache, MySQL, and PHP. LAMP is a widely used stack for constructing web applications.

Figure 1.5: Containers on Top of a Container Runtime Engine on Top of an Operating System on Top of a Hypervisor (or Bare Metal)



The process begins with creating a container image containing a Linux distribution. We'll choose Ubuntu for this example. This image can be downloaded from a library using the container management system. Once you have created the Ubuntu container image and identified it as an image, you execute it, i.e., make a container, and you use that container to load Apache using features of Ubuntu. Now you exit the container and inform the container management system that this is a second image. You execute this second image and load MySQL. Again, you exit the container and give the image a name. Repeating this process one more time and loading PHP, you end up with a container image holding the LAMP stack. Because this image was created in steps and you told the container management system to make each step an image, the final image is considered by the container management system to be made up of layers.

Now you move the LAMP stack container image to a different location for production use. The initial move requires moving all the elements of the stack, so the time it takes is the time to move the total stack. Suppose, however, you update PHP to a newer version and move this revised stack into production. The container management system knows that only PHP was revised and

moves only PHP layer of the image. This saves the movement of the rest of the stack. Since changing a software component within an image happens much more frequently than initial image creation, placing a new version of the container into production becomes a much faster process than it would be using a VM. Whereas loading a VM takes on the order of minutes, loading a new version of a container takes on the order of microseconds or milliseconds.

You can script the creation of a container image through a file. This file is specific to the tool you are using to create the container image. Such a file allows you to specify what pieces of software are to be loaded into the container and saved as an image. Using version control on the specification file ensures that each member of your team can create an identical container image and modify the specification file as needed. We discuss these practices in more detail in Chapter 7 Managing System Configurations.

Sidebar: Container Images

The terminology distinction between a VM and a VM image carries over to containers. That is, a container image is a set of bits that when executed becomes a container.

However, this terminology is not always followed in practice. The term container is frequently used to refer to either a container image or a container, depending on context. For example, in **Error! Reference source not found.** we will discuss container repositories. In fact, a container repository is actually a repository for container images.

Understanding the distinction between containers and container images is important when you use containers but maintaining this distinction when discussing or writing about containers is cumbersome.

1.5 Summary

Hardware virtualization allows the creation of several virtual machines sharing the same physical machine. It does this while enforcing isolation of CPU, memory, disk storage, and network. This allows the resources of the physical machine to be shared among several VMs and reduces the number of physical machines that an organization must purchase or rent.

A virtual machine image is the set of bits that are loaded into a VM to enable its execution. Virtual machine images are created by various techniques for

provisioning including using operating system functions or loading a pre-created image.

Containers are virtualized operating systems and provide performance advantages over VMs. Containers constructed in terms of layers are faster to deploy when a component changes.

1.6 Exercises

Remember when you perform these exercises that the internet is your friend. Begin an exercise by searching for "tutorial [subject]." Someone will likely have written a description of how to use a particular tool or perform a particular operation. If you get an error message when following the tutorial instruction, someone else has likely gotten it and written an explanation. You can find the explanation by pasting the error message into your search engine.

1. Create a virtual manager on your host machine by downloading VirtualBox.
2. Create a virtual machine using VirtualBox running Ubuntu.
3. Load LAMP into your virtual machine using apt-get. What are the errors that you made?
4. Create a Docker container image with the LAMP stack in four layers. Test the image by executing a simple PHP program.

1.7 Discussion Questions

1. Where do you find the Ubuntu distribution? What are the steps that take a Linux kernel distribution and creates an Ubuntu distribution? Who provides the funding for the creation of the Ubuntu distribution?
2. Enumerate different package managers used by the different distributions of Linux. What are the differences between them?
3. Two VMs hosted on the same physical computer are isolated, but it is still possible for one VM to affect the other VM. How can this happen?
4. How does the container management system know that only one layer has been changed so that it needs to transport only one layer?
5. We've focused on isolation among VMs that are running at the same time on a hypervisor. VMs shut down and stop executing, and new

VMs start up. What does a hypervisor do to maintain isolation, or prevent leakage, between virtual machines running at different times

Introduction to Part 2

It's a rainy Monday morning, and you are on your way back to your desk with your first cup of coffee when your boss calls from behind you: "You missed a long weekend! Our cloud provider had a network failure, and our pagers went off. We had to do a manual failover to our backup region. Can you SSH into the service you deployed last week and check the logs? It looked like there might be a problem with log rollover. Talk to Jan to get your certs set up. OK? Also, your new assignment will have external users, so you should get familiar with OAuth." You are pretty sure that your boss doesn't want you to go down to the river and roll logs with the lumberjacks, but you suddenly feel like one cup of coffee isn't going to be enough this morning.

Part 2 deals with things that are under your control—microservice architectures, managing configuration of the deployment pipeline, the things you are responsible for after your service goes into production, disaster recovery, and those aspects of security that you can control.

Chapter 6 discusses microservice architecture. Microservice architectures have a symbiotic relationship with containers. That is, packing a microservice architecture in a container simplifies the deployment of the microservice, and a microservice's size suggests a limited use of resources that fits into the constraints on containers. Another relationship we will explore is that between microservices and team size.

The second portion of our discussion of microservices focuses on the technical aspects of microservices. We discuss microservices from the perspective of their quality attributes—performance, reusability, modifiability, and availability. The relationship between a microservice and its cloud platform plays a role in understanding these quality attributes.

Since a microservice communicates only via messages, the protocols used to pass messages are important. We discuss REST and gRPC since those are commonly used in the internet environment.

The final portion of our microservice discussion deals with the common services used by microservices. These are common to many microservices within an organization and are factored out into libraries or other services. Here is where you will read about discovery and registration.

Chapter 7 introduces configuration management, which is storing and applying changes to the configuration of systems and services. Your configuration is represented as scripts, and we will discuss the tools and practices for the management of systems and configuration parameters.

Chapter 8 discusses the deployment pipeline. Modern practices use continuous delivery, if not continuous deployment. The pipeline is dependent on using different environments for different purposes, and we identify environmental management as a first-class concept. The deployment pipeline is heavily dependent on tools, and we discuss different classes of tools used in a pipeline.

We also discuss different strategies for deployment, including partial deployment. Deployments can be all or nothing, or incremental. There are also several test-deployment strategies—canaries and A/B testing are two that we discuss.

Next, in Chapter 9 Post Production, we discuss system logging and monitoring. These provide visibility into how well your services are executing and should provide warnings, through alerts, before service failures impact your users.

Sometimes, bad things happen that are beyond your control (such as the network connection to your cloud service provider being cut by a backhoe). All the monitoring in the world can't predict these events. This leads to the topic of Chapter 10, disaster recovery, which are the steps we take to be sure that essential services are available for your users, even when these events happen.

Chapter 11 returns to security, which is a major operations concern. In most cases, your production environment is connected to the internet, and we have to take steps to maintain confidentiality of your data (only authorized parties can see it), integrity of your services and data (no unauthorized parties can change them), and availability of your services (authorized users can access your systems when they want to). We also discuss the software supply chain and patching vulnerabilities.

We end with Chapter 12, which provides a brief discussion of some topics that we think are either interesting or important that we didn't find a place for in the first 11 chapters.